# Computer Games Development
# Project Report
# Year IV

[Jad Fuhr]
[C00290965]
[12/04/2026]

**Contents**

Acknowledgements ..... 2
Project Abstract ..... 2
Project Introduction and/or Research Question ..... 2
Literature Review ..... 2
Evaluation and Discussion ..... 3
Conclusions ..... 3
References ..... 4
Appendices ..... 4

## Project Abstract

This project presents the design and implementation of an AI system for PaleoPals, a 2D mining and fossil-collection game built using C++ and SFML 3. The game features a destructible tile-based world, a player-controlled miner, and collectible fossils that contribute to museum exhibits and in-game progression. The original AI system for non-player palaeontologists relied on rigid, state-based logic and simple downward digging behaviour, which limited believability, scalability, and player engagement.

To address these limitations, this project introduces a robust AI architecture combining A* pathfinding with a behaviour tree system. The new system enables palaeontologist agents to navigate a dynamically changing underground environment, mine tiles intelligently, collect fossils, return to the surface, and even follow the player as a support companion. The AI integrates with the game's economic system, allowing players to hire additional palaeontologists whose costs scale with each hire.

The resulting system demonstrates how behaviour trees and A* can be applied effectively in a destructible terrain game, producing agents that behave more intelligently, adapt to environmental changes, and contribute meaningfully to gameplay. The work provides a foundation for future expansions, including more complex agent roles, cooperative behaviours, and improved world simulation.

# Project Introduction and/or Research Question

## Background

PaleoPals is a 2D mining and exploration game where the player digs through layered terrain to uncover fossils, amber, and other collectibles. These items contribute to museum displays, unlock cosmetic skins, and generate money used for upgrades. The world is fully destructible: each tile has hardness, hit points, and crack states, and breaking tiles may spawn collectibles. The player interacts with buildings such as the Museum and Trader which manages upgrades and navigates a large underground map.

Early in development, the game included a basic AI palaeontologist who followed a simple scripted routine: wander, dig straight down, mine tiles, and return to the surface. While functional, this system lacked flexibility and produced repetitive, unnatural behaviour. It also struggled with the destructible grid, often getting stuck or failing to navigate around obstacles.

## Motivation

As the game expanded, it became clear thar more intelligent AI helpers would

- Reduce repetitive tasks for the player
- Make the world feel more alive
- Improve long term engagement and progression

This motivated me to redesign the AI system

## Research Question

How can behaviour trees combined with A* pathfinding be used to create believable autonomous palaeontologist agents in a destructible tile-based mining game?

## Project Objectives

- Implement a reusable A* pathfinding module compatible with destructible terrain.
- Replace the old, scripted AI with a modular behaviour tree system
- Support multiple palaeontologists with different roles
- Integrate AI behaviour with the games economy and progression systems
- Improve navigation reliability in a dynamic underground environment

**Main Contributions**

- A behaviour tree framework enabling modular, readable AI logic
- Two distinct AI roles: Follower and AutoMiner

- Follower who tracks the player and collects missed fossils.
- AutoMiner who searches for fossils, mines independently, and returns to the surface.

- Integration with the existing fossil, mining, and upgrade systems.
- A more dynamic and believable game world.

## Literature Review

### Behaviour Trees in Game AI

Behaviour trees (BTs) are widely used in modern game development due to their modularity, readability, and scalability. Unlike finite state machines, BTs avoid state explosion and allow designers to compose complex behaviours using simple building blocks such as:

- Selectors (choose the first successful child)
- Sequences (execute children in order until one fails)
- Conditions
- Actions

BTs originated in robotics and were popularised in games such as Halo 2 and Spore. Their hierarchical structure makes them ideal for agents that must react to changing environments, such as palaeontologists navigating a destructible mine.

### A* Pathfinding on Tile Based Grids

A* is the industry standard for pathfinding in grid-based games. It uses a heuristic (Manhattan distance in this project) to efficiently find the lowest cost path between two points. A* is particularly effective when:

- The environment is grid based
- Movement cost is uniform
- Obstacles change over time

In PaleoPals, tiles can be destroyed, creating new walkable paths. A* was chosen because:

- It handles dynamic obstacles well
- It produces optimal paths
- It is easy to integrate with behaviour trees
- It is efficient enough for a 75×100 grid

## AI Helpers in Games

Companion AI is common in RPGs, survival games, and automation focused titles. Research shows that helper agents:

- Reduce player workload
- Increase perceived world activity
- Improve pacing and flow
- Provide emotional engagement

Examples include:

- Minions in Terraria
- Dwarves in Dwarf Fortress
- Villagers in Minecraft
- Pawns in RimWorld

These systems influenced the design of palaeontologists as semiautonomous helpers who meaningfully contribute to gameplay.

## Influence on this Project

- BTs were chosen over FSMs due to scalability and clarity.
- A* was selected for its optimality and suitability for destructible grids.
- Helper agent research informed the follower/miner role split.
- Game AI design patterns guided the modular architecture.

## Evaluation and Discussion

### Functional Evaluation

The new AI system was tested across several scenarios:

### Follower

- Successfully tracks the player using A*
- Avoids obstacles and destroyed tiles
- Picks up fossils the player leaves behind
- Maintains a comfortable following distance
- Behaves naturally in both surface and underground environments

### AutoMiner

- Scans for nearby fossils
- Computes A* paths to target tiles
- Mines tiles using the existing hardness/HP system
- Places and removes ladders as needed
- Returns to the surface when inventory is full or task is complete

### Performance

- A* searches are limited to local regions to reduce cost.
- Behaviour trees run every frame, but actions are lightweight.
- No noticeable frame drops with up to 3 palaeontologists active.

### Limitations

- Agents can occasionally choose suboptimal paths in complex tunnels.
- No collision avoidance between palaeontologists yet.
- Behaviour trees could be expanded with more nuanced decision-making.

## Milestones

November 30th:

- Started Paleontologist behaviour implementation

December 31st:

- Completed Paleontologist behaviours
- Museum Screen/Fossil Codex completed
- Fossil collection stored in museum codex
- Trader Screen Implemented
- Researcher Behaviour Started

January 31st:

- Researcher Behaviours completed
- Able to assign researchers to completed skeleton to gain facts on specific species
- Economy system started
- Hiring of more Paleontologists/Researchers and Trader(changes trader screen from market to include upgrades)
- Upgrade system started
- Sonar upgrade mostly completed (main selling point)

February 28th:

- Research system fully implemented
- Economy system fully implemented
- Major upgrades created and functional
- Make map more customisable/versatile (possibly load preloaded maps from JSON[level 1, level 2, etc...])

March 31st:

- Make map more customisable/versatile (possibly load preloaded maps from JSON[level 1, level 2, etc...]) cont.
- Code optimisation/refactoring for efficiency started

April 30th:

- code efficiency increased
- map fully versatile with multiple levels of varying difficulty
- Final touch ups (sprites/code)

It is safe to say that my milestones were not met in the desired time in which I had allotted to them.

A great example of this is me completely scrapping the Researcher AI idea and opting to purely focus on my palaeontologists

## Technical Achievement

- Implemented a full A* pathfinding system for a destructible tile grid.
- Designed and integrated a behaviour tree framework in C++.
- Replaced the old, scripted AI with modular, extensible behaviours.
- Created two AI roles: follower and autonomous miner.
- Integrated AI with the fossil, mining, and economic systems.
- Improved camera, collision, and world space interactions to support AI.
- Ensured compatibility with existing systems (museum, trader, upgrades).

### Review

### What Went Right

- Behaviour trees made AI logic far easier to manage and extend.
- A* provided reliable navigation even in complex underground structures.
- The follower/miner split created engaging emergent behaviour.
- Integration with the economy added meaningful player choices.
- The system remained performant despite dynamic terrain.

### What Went Wrong

- Early AI design was too rigid and had to be scrapped.
- Debugging tile offsets and world to grid conversions took significant time.
- Pathfinding initially struggled with vertical shafts until ladder logic was added.
- Some UI elements needed rework due to camera zoom and Fullscreen issues.

### Outstanding Work

- Collision avoidance between multiple palaeontologists.
- More sophisticated task allocation (e.g. allocating miners to a job board).
- Better fossil prioritisation (value based or rarity based).
- More complex behaviours (e.g., scouting, placing scaffolding to support tunnels).
- Hazards in the map such as sink holes or collapsing tunnels

### If I Could Start Again

- Build the AI architecture earlier in development.
- Separate navigation grid from rendering logic from the start.
- Plan UI scaling and camera behaviour earlier.
- Basically, do everything a lot earlier and dedicate way more time than I did in general especially when it comes to the harder technological achievement things like the AI behaviour

### Tech Choices

- C++ + SFML: Good performance and control, correct choice.
- Behaviour Trees: Excellent for modular AI, scalable and readable.
- A*: Ideal for grid navigation, robust and well understood.

## Conclusion

This project successfully demonstrates how behaviour trees and A* pathfinding can be combined to create intelligent, believable AI agents in a destructible 2D mining game. The new palaeontologist AI is significantly more capable than the original system, offering improved navigation, autonomous decision-making, and meaningful contributions to gameplay.

The work answers the research question by showing that behaviour trees provide a flexible structure for agent logic, while A* ensures reliable navigation in a dynamic environment. The resulting system enhances player experience, increases world activity, and lays a strong foundation for future expansion.

## Future Work

- More advanced pathfinding (e.g. Jump Point Search, hierarchical A*).
- Player commands (e.g. “mine here”, “follow me”, “return to base”).
- Improved UI for managing hired palaeontologists.
- Procedural fossil distribution and dynamic world events.
- Machine learning approaches for adaptive behaviour.

# References


### Videos Used

Ögren, P. (2021). Behavior trees playlist [YouTube playlist]. YouTube. https://www.youtube.com/playlist?list=PLFQdM4LOGDr_vYJuo8YTRcmv3FrwczdKg

Computerphile. (2014, December 18). A search algorithm* [Video]. YouTube. https://www.youtube.com/watch?v=ySN5Wnu88nE

Natsu Games. (2023). A pathfinding explained simply* [Video]. YouTube. https://www.youtube.com/watch?v=pXsPj6HewhM

Shah, M. (2020). C++ programming playlist [YouTube playlist]. YouTube. https://www.youtube.com/playlist?list=PLvv0ScY6vfd8j-tlhYVPYgiIyXduu6m-L

### Book/s Used

Meyers, S. (1996). *More effective C++: 35 new ways to improve your programs and designs*. Addison-Wesley. (redundant)

### Documentation Used

SFML Team. (2024). SFML 3.0.0 documentation: Functions (I). SFML. https://www.sfml-dev.org/documentation/3.0.0/functions_func_i.html

### Websites Used

Nlohmann. (2024). basic_json API documentation. JSON for Modern C++. https://json.nlohmann.me/api/basic_json/